✅ Task 17: Inter-Service Communication using REST

🎯 Aim

To design and implement REST-based inter-service communication between microservices using Spring Boot, where one microservice consumes APIs from another and handles responses and failures gracefully.

🧩 Problem Statement

In a distributed microservices architecture, services often need to communicate with each other.

Design two microservices:

- **Student Service** → Provides student data (with database)
- **Course Service** → Consumes Student Service APIs using REST

The system must:

- Fetch student details from another service
- Handle failures (service down / timeout)
- Provide fallback response

📐 Architecture Overview

Client → Course Service → Student Service → Database

⚙️ Tools & Technologies

- Spring Boot
- Spring Web
- Spring Data JPA
- SQL Server / H2 Database
- RestTemplate / WebClient
- Eureka (optional but recommended)
- Resilience4j (for fault tolerance)

🚀 STEP-BY-STEP IMPLEMENTATION

✅ 🔹 STEP 1: Create Student Microservice (SQL Server Version)

🎯 Goal

Create a Spring Boot microservice (student-service) that:

- Connects to **SQL Server Express**

- Stores and retrieves student data
- Exposes REST API /students

🧱 STEP 1.1: Create Spring Boot Project (STS)

👉 Open Spring Tools Suite (STS)

1. Go to
   File → New → Spring Starter Project
2. Fill details:

Field	Value
Name	student-service
Type	Maven
Java Version	17
Packaging	Jar
Group	com.example
Artifact	student-service

3. Click **Next**

📦 Dependencies (IMPORTANT)

Select:

- ✅ Spring Web
- ✅ Spring Data JPA
- ✅ SQL Server Driver
- ✅ Lombok

Click **Finish**

🧱 STEP 1.2: Configure SQL Server Database

🔹 1. Open SQL Server Management Studio (SSMS)

Make sure:

- Server: .\SQLEXPRESS
- Login: studentuser
- Password: (your password)

🔹 2. Create Database

```
CREATE DATABASE studentdb;
GO
```

🔹 **3. Use Database**

```
USE studentdb;
GO
```

🔹 **4. Create Table**

```
CREATE TABLE student (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(100)
);
```

🔹 **5. Insert Sample Data**

```
INSERT INTO student (name, department) VALUES ('John', 'AI');
INSERT INTO student (name, department) VALUES ('Alice', 'DS');
```

🧱 STEP 1.3: Configure application.properties

📁 Path:
src/main/resources/application.properties

✅ SQL Server Configuration

```
server.port=8081
spring.application.name=student-service

# SQL Server Connection
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=studentdb;encrypt=false
spring.datasource.username=studentuser
spring.datasource.password=your_password

spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

# JPA Settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

⚠️ Important Notes

- Replace your_password
- Ensure:

- SQL Server is running
- TCP/IP enabled
- Port = 1433

🧱 STEP 1.4: Create Package Structure

Inside src/main/java:

```
com.example.studentservice
    ├── controller
    ├── service
    ├── repository
    └── entity
```

🧱 STEP 1.5: Create Entity Class

📁 entity/Student.java

```
package com.example.studentservice.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String department;

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}
```

🧱 STEP 1.6: Create Repository

📁 repository/StudentRepository.java

```
package com.example.studentservice.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.studentservice.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

🧱 STEP 1.7: Create Service Layer

📁 service/StudentService.java

```
package com.example.studentservice.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.studentservice.repository.StudentRepository;
import com.example.studentservice.entity.Student;

import java.util.List;

@Service
public class StudentService {

    @Autowired
    private StudentRepository repo;

    public List<Student> getAllStudents() {
        return repo.findAll();
    }
}
```

🧱 STEP 1.8: Create Controller

📁 controller/StudentController.java

```
package com.example.studentservice.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.example.studentservice.service.StudentService;
import com.example.studentservice.entity.Student;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {
```

```
    @Autowired
    private StudentService service;

    @GetMapping
    public List<Student> getStudents() {
        return service.getAllStudents();
    }
}
```

🧱 STEP 1.9: Run Application

👉 Right click project → **Run As → Spring Boot App**

🌐 STEP 1.10: Test API

Open browser or Postman:
http://localhost:8081/students

📊 Expected Output

```
[
  {
    "id": 1,
    "name": "John",
    "department": "AI"
  },
  {
    "id": 2,
    "name": "Alice",
    "department": "DS"
  }
]
```

🧪 STEP 1.11: Verification Checklist

✅ SQL Server running
✅ studentdb created
✅ Table student exists
✅ Data inserted
✅ Application runs without errors
✅ API returns JSON

⚠️ Common Errors & Fixes

❌ Error: *Error 26 – Cannot connect to SQL Server*

✓ Fix:

- Enable TCP/IP in SQL Server Configuration Manager
- Restart SQL Server

❌ Error: *Login failed*

✓ Fix:

- Check username/password
- Ensure SQL Authentication enabled

❌ Error: *Port issue*

✓ Fix:

- Ensure port = 1433
- Add in connection string if needed

🎉 Result

✓ Successfully created **Student Microservice**
✓ Connected Spring Boot with **SQL Server Express**
✓ Exposed REST API /students
✓ Verified database integration

✅ 🔹 STEP 3: Create Course Microservice (Consumer)

🎯 Goal

Create a microservice (course-service) that:

- Calls **Student Service (http://localhost:8081/students)**
- Consumes REST API using **RestTemplate**
- Handles failures gracefully
- (Optional) Uses **Resilience4j Circuit Breaker**

🧱 STEP 3.1: Create Spring Boot Project (STS)

👉 Open Spring Tools Suite (STS)

1. Go to:
   File → New → Spring Starter Project

2. Enter details:

Field	Value
Name	course-service
Type	Maven
Java Version	17
Group	com.example
Artifact	course-service

3. Click **Next**

📦 Dependencies

Select:

- ✅ Spring Web
- ✅ Spring Boot DevTools (optional)
- ❗ (Optional) Resilience4j

Click **Finish**

🧱 STEP 3.2: Verify Project Structure

After creation:

```
course-service
└── src/main/java
    └── com.example.courseservice
```

🧱 STEP 3.3: Create Package Structure

Inside:
com.example.courseservice

Create packages:
controller
service
config

👉 Same method as previous:
Right-click → New → Package

🧱 STEP 3.4: Configure application.properties

📁 Path:
src/main/resources/application.properties

✅ **Add:**
```
server.port=8082
spring.application.name=course-service
```

🧱 STEP 3.5: Create RestTemplate Configuration

📁 config/AppConfig.java

👉 **Steps:**

1. Right-click config
2. New → Class → AppConfig

✅ **Code:**
```
package com.example.courseservice.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

🧱 STEP 3.6: Create Service Layer (Consumer Logic)

📁 service/CourseService.java

👉 **Steps:**

1. Right-click service
2. New → Class → CourseService

✅ **Code:**

```
package com.example.courseservice.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class CourseService {

    @Autowired
    private RestTemplate restTemplate;

    private final String STUDENT_URL = "http://localhost:8081/students";

    public Object getStudentData() {
        try {
            return restTemplate.getForObject(STUDENT_URL, Object.class);
        } catch (Exception e) {
            return "Student Service is currently unavailable. Please try later.";
        }
    }
}
```

🧱 **STEP 3.7: Create Controller**

📁 controller/CourseController.java

👉 **Steps:**

1. Right-click controller
2. New → Class → CourseController

✅ **Code:**

```
package com.example.courseservice.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.example.courseservice.service.CourseService;

@RestController
@RequestMapping("/courses")
```

```
public class CourseController {

    @Autowired
    private CourseService service;

    @GetMapping("/students")
    public Object fetchStudents() {
        return service.getStudentData();
    }
}
```

🧱 STEP 3.8: Run Both Microservices

🔹 1. Run Student Service FIRST

👉 Port:
8081

🔹 2. Run Course Service

👉 Port:
8082

🌐 STEP 3.9: Test API

👉 Open Browser / Postman:
http://localhost:8082/courses/students

📊 OUTPUT

✅ Case 1: Student Service is RUNNING


```
[
  {"id":1,"name":"John","department":"AI"},
  {"id":2,"name":"Alice","department":"DS"}
]
```

❌ Case 2: Student Service is STOPPED
Student Service is currently unavailable. Please try later.

🚀 🔹 STEP 4: Add Resilience4j (Advanced Fault Tolerance)

🧱 STEP 4.1: Add Dependency (pom.xml)

```
<dependency>
   <groupId>io.github.resilience4j</groupId>
   <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

🧱 STEP 4.2: Enable Circuit Breaker

📁 CourseService.java

✅ Updated Code:

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;

@Service
public class CourseService {

   @Autowired
   private RestTemplate restTemplate;

   private final String STUDENT_URL = "http://localhost:8081/students";

   @CircuitBreaker(name = "studentService", fallbackMethod = "fallbackMethod")
   public Object getStudentData() {
      return restTemplate.getForObject(STUDENT_URL, Object.class);
   }

   public Object fallbackMethod(Exception e) {
      return "Fallback: Student Service Down!";
   }
}
```

🧱 STEP 4.3: Add Config (Optional but Good Practice)

📁 application.properties

```
resilience4j.circuitbreaker.instances.studentService.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.studentService.sliding-window-size=10
```

🧪 STEP 5: Final Testing

✅ When Service is UP:

http://localhost:8082/courses/students

✓ Returns student data

❌ When Service is DOWN:

✓ Returns:
Fallback: Student Service Down!

🧪 Verification Checklist

✓ Student service running (8081)
✓ Course service running (8082)
✓ API call successful
✓ Failure handled properly
✓ No application crash

⚠️ Common Errors & Fixes

❌ 404 Error

✓ Check URL:
/courses/students

❌ Connection Refused

✓ Ensure:

- Student service running
- Port = 8081

❌ Bean Not Found (RestTemplate)

✓ Ensure:
@Bean
public RestTemplate restTemplate()

❌ CircuitBreaker not working

✓ Add dependency correctly
✓ Clean & Build project

🎉 Final Result

✓ Implemented **Inter-Service Communication (REST)**
✓ Course Service consumes Student Service
✓ Implemented **graceful error handling**
✓ Added **fault tolerance using Circuit Breaker**

🔥 Architecture Summary

Client → Course Service → Student Service → SQL Server DB

📈 Result

- Successfully implemented **REST-based inter-service communication**
- Course Service consumes Student Service API
- Implemented **fault tolerance using try-catch / Circuit Breaker**
- Ensured **graceful degradation during failures**

💡 Real-Time Applications

- E-commerce (Order Service ↔ Product Service)
- Banking (Account ↔ Transaction Service)
- Healthcare (Patient ↔ Appointment Service)
- Cloud-based distributed systems